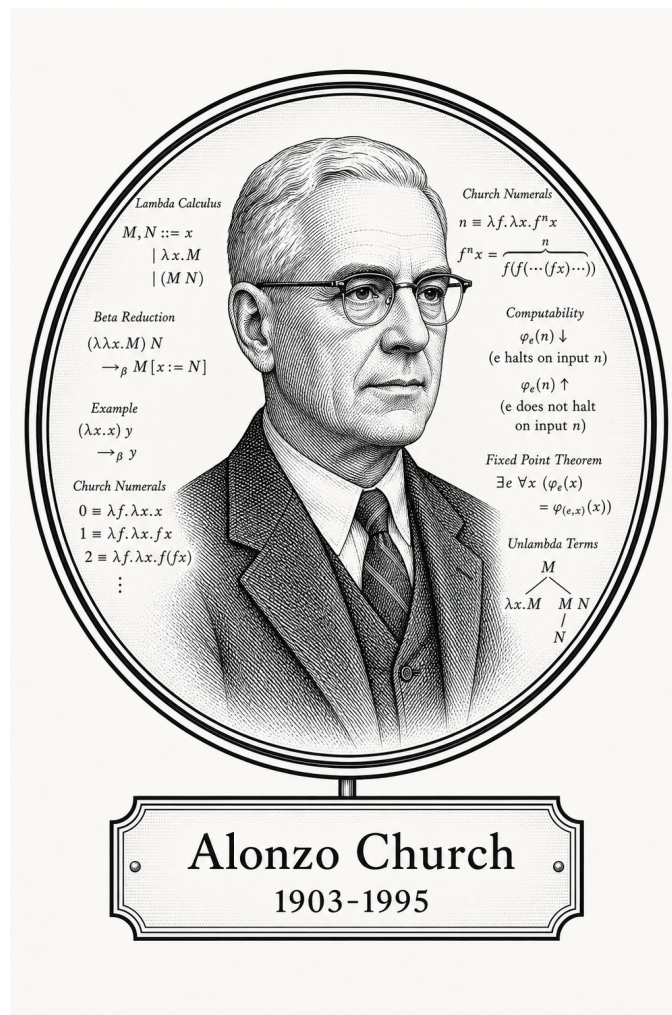


FROM CANTOR TO ITO

Advanced Logic

Lambda Calculus



Alonzo Church

1903-1995

Self-Study Notes

William Sollers

July 1, 2026

For every reader learning to make mathematics their own.

Contents

1	Lambda Calculus	1
1.1	Lambda Terms	1

Chapter 1

Lambda Calculus



1.1 Lambda Terms

Section Toolkit — Quick Reference

Concept	One-line summary
λ -term	Variable x , abstraction $\lambda x.M$, application MN
α -equivalence	$\lambda x.M =_\alpha \lambda y.M[y/x]$ when y fresh
β -reduction	$(\lambda x.M)N \rightarrow_\beta M[N/x]$
Normal form	Term with no β -redex; may not exist (non-termination)
Church boolean T	$\lambda x.\lambda y.x$
Church boolean F	$\lambda x.\lambda y.y$
Church numeral $\bar{n}$	$\lambda f.\lambda x.f^n(x)$ (f applied n times to x)
Type $\tau \rightarrow \sigma$	Function type: input of type τ , output of type σ
Typing judgment	$\Gamma \vdash M : \tau$ reads “ M has type τ in context Γ ”
Curry–Howard	Propositions are types; proofs are terms; $A \rightarrow B$ is implication

Untyped Lambda Calculus

Background References

- **First-order logic:** Enderton [1972], Barwise [1977a].
- **General topology:** Kelly [1955].
- **Set theory:** Halmos [1960], van Dalen et al. [1978].
- **Recursion theory:** Rogers [1967].
- **Category theory:** Arbib and Manes [1975], MacLane [1972].

Definition — Lambda Terms

Definition 1.1.1 (Lambda Terms). The set Λ of λ -terms is defined inductively as follows:

1. Every variable x is a λ -term.
2. If M is a λ -term and x is a variable, then

$$\lambda x. M$$

is a λ -term.

3. If M and N are λ -terms, then

$$MN$$

is a λ -term.

Remark (Standard quantified statement).

$M \in \Lambda \iff M$ is generated from variables by finitely many applications of $\lambda x.(\cdot)$ and $(\cdot)(\cdot)$. ■

Remark (Interpretation). The λ -terms are the smallest set closed under three constructions: variables, abstraction (function formation), and application (function evaluation). This inductive definition plays the same role as recursive definitions in analysis or algebra: every λ -term is built from variables by finitely many applications of abstraction and application.

Remark (Structural View). Every λ -term has a tree structure:

- variables are leaves,
- abstractions $\lambda x. M$ are unary nodes,
- applications MN are binary nodes.

Thus Λ is a freely generated syntactic algebra.

Bibliography